UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Sweet Tapper Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Sweet Tapper

Sweet Tapper is a standalone editable Unity game template. Tap the large treat to bake treats. Buy Bakers for automatic production and upgrades for stronger output. Fast tapping heats the oven into a Hot Streak; catch golden cupcakes and follow chapter goals.

Includes 25 curated chapters, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Additional included libraries: 12 producers, 40 upgrades, 26 achievements, 4 boosters. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Sweet Tapper > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/SweetTapper/Scenes/SweetTapper_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/SweetTapper`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/sweettapper contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/StArt.cs`
- `Scripts/StData.cs`
- `Scripts/StEcon.cs`
- `Scripts/StFx.cs`
- `Scripts/StGen.cs`
- `Scripts/StLevels.cs`
- `Scripts/StSfx.cs`
- `Scripts/StSim.cs`
- `Scripts/SweetTapperGame.cs`
- `Scripts/SweetTapperGame.Layout.cs`
- `Scripts/SweetTapperGame.Popups.cs`

Main flow: SweetTapperGame.cs. UI and popup consumers: SweetTapperGame.Popups.cs. Model/generator/loading: StData.cs, StEcon.cs, StGen.cs, StLevels.cs, StSim.cs. Authoring: Editor/SweetTapperLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Tap the large treat to bake treats. Buy Bakers for automatic production and upgrades for stronger output. Fast tapping heats the oven into a Hot Streak; catch golden cupcakes and follow chapter goals.

The initial bank has 25 curated chapters. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact helper names, unlocks, charges and effects: Resources/sweettapper/Levels/levels.json / boosters.

Helper	Unlock	Free charges	Action and effect
Sugar Rush	4	3	Taps bake x7 for 30 seconds.
Golden Hour	6	2	Three golden cupcakes float by over the next few seconds.
Whisk Bot	8	2	A little robot taps 12 times a second for 45 seconds.
Oven Timer	10	1	Instantly bake ten minutes of production.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Sweet Tapper > Level Editor**. Exact implementation: `Editor/SweetTapperLevelEditor.cs`.

Duplicate an early entry in this game-specific visual editor, change its visible name and one rule, save, validate, then play it. Use the exact source-control table below for this game.

1. Back up Resources/sweettapper/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`SweetTapperLevelEditor.cs:53`	`[MenuItem("Szekipapa77/Sweet Tapper/Level Editor")]`
`SweetTapperLevelEditor.cs:56`	`[MenuItem("Szekipapa77/Sweet Tapper/Validate Economy")]`
`SweetTapperLevelEditor.cs:126`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`SweetTapperLevelEditor.cs:127`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`SweetTapperLevelEditor.cs:130`	`if (GUILayout.Button("Validate (bots)", EditorStyles.toolbarButton, GUILayout.Width(100))) RunValidation();`
`SweetTapperLevelEditor.cs:133`	`if (_section == Section.Chapters && GUILayout.Button(new GUIContent("Quick test", "Optional: saves, opens the WebGL scene and starts at this chapter with a bot-played state. Editing never needs Play mode."), EditorStyles.toolbarButton, GUILayout.Width(80))) Play(_sel + 1);`

SweetTapperLevelEditor.cs:190	if (arr == null) { EditorGUILayout.HelpBox("Tuning and endless-generator settings.", MessageType.Info); EditorGUILayout.EndVertical(); return; }
SweetTapperLevelEditor.cs:207	if (GUILayout.Button("Add")) Insert(_sel + 1, NewItem());
SweetTapperLevelEditor.cs:208	if (GUILayout.Button("Dup") && arr.Length > 0) Insert(_sel + 1, CloneItem(arr.GetValue(_sel)));
SweetTapperLevelEditor.cs:209	if (GUILayout.Button("Del") && arr.Length > 0 && EditorUtility.DisplayDialog("Delete", "Delete " + ItemLabel(_sel) + "?", "Delete", "Cancel"))
SweetTapperLevelEditor.cs:213	if (GUILayout.Button("Up") && _sel > 0) MoveSel(-1);
SweetTapperLevelEditor.cs:214	if (GUILayout.Button("Down") && _sel < arr.Length - 1) MoveSel(1);
SweetTapperLevelEditor.cs:274	int n = EditorGUILayout.Popup(label, idx, opts.Select(s => s == "" ? "(none)" : s).ToArray());
SweetTapperLevelEditor.cs:277	else if (f.Name == "story" || f.Name == "desc") f.SetValue(o, EditorGUILayout.TextField(label, (string)v ?? ""));
SweetTapperLevelEditor.cs:278	else f.SetValue(o, EditorGUILayout.TextField(label, (string)v ?? ""));
SweetTapperLevelEditor.cs:285	else if (booster) { var names = new[] { "(none)" }.Concat(_data.boosters.Select(b => b.name)).ToArray(); f.SetValue(o, EditorGUILayout.Popup(label, (int)v + 1, names) - 1); }
SweetTapperLevelEditor.cs:286	else if (f.Name == "area") { var names = new[] { "(keep)" }.Concat(_data.tuning.areaNames).ToArray(); f.SetValue(o, EditorGUILayout.Popup(label, (int)v + 1, names) - 1); }

SweetTapperLevelEditor.cs:287	else if (f.Name == "stage") { var names = new[] { "(keep)" }.Concat(_data.tuning.stageNames).ToArray(); f.SetValue(o, EditorGUILayout.Popup("Treat stage on complete", (int)v + 1, names) - 1); }
SweetTapperLevelEditor.cs:320	if (arr.Length == 0) { EditorGUILayout.HelpBox("Empty list - press Add.", MessageType.Info); return; }
SweetTapperLevelEditor.cs:327	EditorGUILayout.LabelField("Tuning (taps, golden cupcakes, heat, offline, prestige, feature chapters)", EditorStyles.boldLabel);
SweetTapperLevelEditor.cs:330	EditorGUILayout.LabelField("Endless generator (chapters after the curated list; the game runs the same StGen code)", EditorStyles.boldLabel);
SweetTapperLevelEditor.cs:333	EditorGUILayout.LabelField("Preview of the next generated chapters (relative goals are instantiated from the player's state):", EditorStyles.miniBoldLabel);
SweetTapperLevelEditor.cs:338	EditorGUILayout.LabelField($"{n}. {c.name} {(c.breather ? " (breather)" : "")}", e.GoalText(c) + $" reward {StNum.Time(c.rewardSeconds)} + fame {c.fame}");
SweetTapperLevelEditor.cs:347	EditorGUILayout.LabelField("Producer ladder - log10 scale. Orange = base cost, green = output/s, blue bars = payback seconds. Drag a point to edit; click to select.", EditorStyles.miniBoldLabel);
SweetTapperLevelEditor.cs:357	EditorGUILayout.LabelField($"{p.name}: payback {StGen.Payback(p):0}s, cost x{(_sel > 0 ? p.baseCost / ps[_sel - 1].baseCost : 1):0.0} over the previous tier", EditorStyles.boldLabel);
SweetTapperLevelEditor.cs:361	_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate producer table (curve controls; keeps names, art and unlocks)", true);
SweetTapperLevelEditor.cs:364	_gFirstCost = EditorGUILayout.DoubleField("First cost", _gFirstCost);
SweetTapperLevelEditor.cs:365	_gCostRatio = EditorGUILayout.Slider("Cost ratio per tier", (float)_gCostRatio, 3, 20);

SweetTapperLevelEditor.cs:366	`_gFirstPayback = EditorGUILayout.Slider("First payback (s)", (float)_gFirstPayback, 5, 300);`
SweetTapperLevelEditor.cs:367	`_gPaybackGrowth = EditorGUILayout.Slider("Payback growth per tier", (float)_gPaybackGrowth, 1, 2.5f);`
SweetTapperLevelEditor.cs:368	`_gGrowth = EditorGUILayout.Slider("Price growth per purchase", _gGrowth, 1.05f, 1.3f);`
SweetTapperLevelEditor.cs:369	`if (GUILayout.Button("Generate producer table", GUILayout.Height(24)))`
SweetTapperLevelEditor.cs:446	`EditorGUILayout.LabelField("Recipe costs (log10) by when they become available (chapter / requirement). Drag a point to change its cost.", EditorStyles.miniBoldLabel);`
SweetTapperLevelEditor.cs:456	`EditorGUILayout.LabelField(u.name + ": " + StEcon.EffectText(_data, u), EditorStyles.boldLabel);`
SweetTapperLevelEditor.cs:481	`EditorGUILayout.LabelField($"Chapter {_sel + 1}: {c.name}", EditorStyles.boldLabel);`
SweetTapperLevelEditor.cs:483	`EditorGUILayout.LabelField("Goal", e.GoalText(c) + (c.relative ? " (relative: set when the chapter starts)" : ""), EditorStyles.helpBox);`
SweetTapperLevelEditor.cs:484	`foreach (var p in _data.producers) if (p.unlockChapter == _sel + 1) EditorGUILayout.LabelField("Unlocks", "baker " + p.name);`
SweetTapperLevelEditor.cs:485	`foreach (var b in _data.boosters) if (b.unlockChapter == _sel + 1) EditorGUILayout.LabelField("Unlocks", "helper " + b.name);`
SweetTapperLevelEditor.cs:487	`EditorGUILayout.LabelField("Measured", $"optimal {c.minutes:0.0} min, casual {c.casualMinutes:0.0} min in this chapter (Validate)");`
SweetTapperLevelEditor.cs:494	`EditorGUILayout.LabelField("Time per chapter (log minutes): orange = optimal bot, purple = casual bot, line = target curve (casual), blue = breather. Click a bar to select.", EditorStyles.miniBoldLabel);`

SweetTapperLevelEditor.cs:526	`EditorGUILayout.LabelField("Auto Generate", EditorStyles.boldLabel);`
SweetTapperLevelEditor.cs:528	`_cFrom = EditorGUILayout.FloatField("Start minutes", _cFrom); _cTo = EditorGUILayout.FloatField("End minutes", _cTo);`
SweetTapperLevelEditor.cs:529	`_cBreather = EditorGUILayout.IntSlider("Breather every", _cBreather, 0, 8);`
SweetTapperLevelEditor.cs:531	`if (GUILayout.Button("Retune all goal values to the target curve (casual bot: 5 taps/s, cheapest useful buys; same StEcon as the game)", GUILayout.Height(24))) Retune();`
SweetTapperLevelEditor.cs:533	`_appendCount = EditorGUILayout.IntSlider("Append generated chapters", _appendCount, 1, 20);`
SweetTapperLevelEditor.cs:534	`if (GUILayout.Button("Append (seeded StGen)", GUILayout.Width(180)))`
SweetTapperLevelEditor.cs:587	`if (_opt == null) { EditorGUILayout.HelpBox("Press Validate to run the optimal and casual player bots and see the time-to-milestone curves.", MessageType.None); return; }`
SweetTapperLevelEditor.cs:589	`EditorGUILayout.LabelField("Validation (bots play the real economy)", EditorStyles.boldLabel);`

CHAPTER 07

Generator, insertion and replacement

Use this editor generation panel, choose a reproducible seed and small count, review start/index and replace behavior, then generate, save and reopen. The source excerpt below identifies exact append/replace semantics.

Source	Exact current code

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/SweetTapper/Resources/sweettapper/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "name": "The First Batch",
  "story": "Welcome to your little kitchen! Tap the cookie to bake your very first treats.",
  "goal": "bake",
  "target": 0,
  "value": 150,
  "hint": "tap",
  "area": 0,
  "stage": -1,
  "rewardSeconds": 0,
  "rewardFlat": 10,
  "rewardBooster": -1,
  "rewardCharges": 0,
  "fame": 0,
  "breather": false,
  "relative": false,
  "minutes": 0.35833332,
  "casualMinutes": 0.5
}
```


Root keys: version, producers, upgrades, chapters, achievements, boosters, tuning, gen. Inspect StData.cs, StEcon.cs, StGen.cs, StLevels.cs, StSim.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/StArt.cs`. Find `Cream`, currently `#FFF4E0`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`SweetTapperGame.Layout.cs:183 / void BuildGoalCard(float cx, float cy, float w, float h)`	`var card = HzUI.Panel("GoalCard", _layer, Cream, 40);`
`SweetTapperGame.Layout.cs:419 / void SetTab(int t, bool animate = true)`	`_tabImgs[i].color = sel ? HzUI.Hex(TabColors[i]) : Color.Lerp(HzUI.Hex(TabColors[i]), Cream, 0.62f);`

Source	Exact current code
`StArt.cs:9 / class/static declarations`	`public static readonly Color Cream = HzUI.Hex("#FFF4E0"), CreamDeep = HzUI.Hex("#F7E3C4"), Strawberry = HzUI.Hex("#FF7A9C"), Caramel = HzUI.Hex("#E8A04C"),`
`StArt.cs:10 / class/static declarations`	`Mint = HzUI.Hex("#6FC3A8"), Cocoa = HzUI.Hex("#6B3E2E"), CocoaSoft = HzUI.Hex("#9A6A55"), Plum = HzUI.Hex("#3B2A4A"), Gold = HzUI.Hex("#FFC94A"),`
`StArt.cs:11 / class/static declarations`	`Berry = HzUI.Hex("#E2557A"), Sky = HzUI.Hex("#7FB7E8"), Lilac = HzUI.Hex("#A88BE0"), Fire = HzUI.Hex("#FF7B3A"), Locked = HzUI.Hex("#B9A898"),`
`StArt.cs:12 / class/static declarations`	`Ink = HzUI.Hex("#5A3426"), Sheet = HzUI.Hex("#FFF8EC"), RowBg = HzUI.Hex("#FFFFFF"), RowDim = HzUI.Hex("#F3E6D3"), Good = HzUI.Hex("#4FB46E");`

StArt.cs:13 / class/static declarations	public static readonly string[] Sprinkles = { "#FF7A9C", "#FFC94A", "#6FC3A8", "#7FB7E8", "#A88BE0", "#FFFFFF", "#E8A04C" };
StArt.cs:16 / class/static declarations	public static readonly string[] AreaTop = { "#F9D9B5", "#FFD3C9", "#CFEFD9", "#F6C99A", "#4B3A78" };
StArt.cs:17 / class/static declarations	public static readonly string[] AreaBottom = { "#E9A97A", "#E79AA8", "#8FCFAE", "#C9805A", "#1E1838" };
StData.cs:21 / class/static declarations	public string color = "#E8A04C";
StData.cs:103 / class/static declarations	public string color = "#FF7A9C";
SweetTapperGame.cs:200 / void DoTap(Vector2 local, bool player)	Color c = hot ? HzUI.Hex("#FFD27A") : rush ? HzUI.Hex("#FFC2D4") : Color.white;
SweetTapperGame.cs:233 / void OnStreakStart()	Say("HOT STREAK! Taps x" + D.tuning.streakMult.ToString("0", Inv), HzUI.Hex("#FFD27A"), 1.3f, 0.62f);
SweetTapperGame.cs:372 / void CatchGolden(Golden g)	Say("Oven Frenzy! Production x" + D.tuning.frenzyMult.ToString("0", Inv) + " for " + D.tuning.frenzySeconds.ToString("0", Inv) + "s", HzUI.Hex("#FFB36B"), 1.8f, 0.62f);
SweetTapperGame.cs:375 / void CatchGolden(Golden g)	Say("Sprinkle Storm! Tap the falling sprinkles!", HzUI.Hex("#FFC2D4"), 1.8f, 0.62f);
SweetTapperGame.cs:435 / void OnBooster(int b)	Say("Sugar Rush! Taps x" + bd.power.ToString("0", Inv) + " for " + bd.duration.ToString("0", Inv) + "s", HzUI.Hex("#FFC2D4"), 1.6f, 0.62f);
SweetTapperGame.cs:447 / void OnBooster(int b)	Say("Whisk Bot is tapping for you!", HzUI.Hex("#BFF0DF"), 1.6f, 0.62f);
SweetTapperGame.Layout.cs:51 / class/static declarations	static readonly string[] TabColors = { "#E8A04C", "#FF7A9C", "#6FC3A8", "#A88BE0" };
SweetTapperGame.Layout.cs:175 / void BuildCounter(float cx, float cy, float w, float fontPx)	_cpsTxt = HzUI.Label("Cps", grp, "", (int)(fontPx * 0.4f), HzUI.Hex("#FFE9C9"), TextAnchor.MiddleCenter, false);
SweetTapperGame.Layout.cs:190 / void BuildGoalCard(float cx, float cy, float w, float h)	var ring = HzUI.Panel("Av", card.transform, HzUI.Hex("#FFE3EA"), (int)(av / 2)); ring.raycastTarget = false;

SweetTapperGame.Layout.cs:203 / void BuildGoalCard(float cx, float cy, float w, float h)	var back = HzUI.Panel("Bar", card.transform, HzUI.Hex("#F0D9BC"), 16); back.raycastTarget = false;
SweetTapperGame.Layout.cs:261 / void BuildHeat(float cx, float cy, float w)	var fl = HzIcons.Icon(bar.transform, IcoFire, 44, HzUI.Hex("#FFB36B"));
SweetTapperGame.Layout.cs:264 / void BuildHeat(float cx, float cy, float w)	_heatFill = HzUI.Panel("Fill", bar.transform, HzUI.Hex("#FFB36B"), 20); _heatFill.raycastTarget = false;
SweetTapperGame.Layout.cs:335 / void RefreshBoosters()	v.Ring.color = _guide == b ? HzUI.Hex("#FFE066") : new Color(1, 1, 1, 0);
SweetTapperGame.Layout.cs:353 / void UpdateBoosterTimer(int b)	v.Caption.color = act ? Gold : E.BoosterUnlocked(b) ? Color.white : HzUI.Hex("#F3E1CF");
SweetTapperGame.Layout.cs:384 / void BuildSheet(float x, float y, float w, float h, float pad, bool portrait)	var bml = HzUI.Label("Buy", _buyModeBtn, "BUY", 22, HzUI.Hex("#F7D9B9")); bml.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(buyW, 30), new Vector2(0, -16));
SweetTapperGame.Layout.cs:558 / float InfoRow(float y, float w, string text)	var iconBack = HzUI.Panel("IcBack", r.rt, HzUI.Hex("#FFF1DD"), (int)(ic * 0.3f)); iconBack.raycastTarget = false;
SweetTapperGame.Layout.cs:562 / float InfoRow(float y, float w, string text)	r.icon = HzUI.Img("Icon", iconBack.transform, art ? art : HzUI.Circle(48), art ? Color.white : HzUI.Hex(tab == 0 ? D.producers[idx].color : "#E8A04C"));
SweetTapperGame.Layout.cs:612 / void UpdateProducerRow(Row r)	r.btnImg.color = can ? Good : HzUI.Hex("#C9B6A4");
SweetTapperGame.Layout.cs:642 / void UpdateUpgradeRow(Row r)	r.btnImg.color = can ? Good : HzUI.Hex("#C9B6A4");
SweetTapperGame.Layout.cs:655 / float BuildGoalsTab(float w)	var card = HzUI.Panel("Current", _content, HzUI.Hex("#FFE9EF"), 34);
SweetTapperGame.Layout.cs:665 / float BuildGoalsTab(float w)	var back = HzUI.Panel("Bar", crt, HzUI.Hex("#F6CFD9"), 16); back.raycastTarget = false;
SweetTapperGame.Layout.cs:685 / float BuildGoalsTab(float w)	var row = HzUI.Panel("Chap" + n, _content, cur ? HzUI.Hex("#FFF4C8") : done ? RowBg : RowDim, 24); row.raycastTarget = false;

`SweetTapperGame.Layout.cs:706 / float BuildGoalsTab(float w)`	`var cell = HzUI.Panel("Ach" + i, _content, got ? HzUI.Hex("#FFF4C8") : RowDim, 22); cell.raycastTarget = false;`
`SweetTapperGame.Layout.cs:743 / float BuildStarsTab(float w)`	`var card = HzUI.Panel("Stars", _content, HzUI.Hex("#F1EAFF"), 34); card.raycastTarget = false;`
`SweetTapperGame.Layout.cs:751 / float BuildStarsTab(float w)`	`_starsSubTxt = HzUI.Label("S", crt, "", 30, HzUI.Hex("#6E5A8A"), TextAnchor.MiddleCenter, false); _starsSubTxt.fontStyle = FontStyle.Bold;`
`SweetTapperGame.Layout.cs:754 / float BuildStarsTab(float w)`	`_nextStarTxt = HzUI.Label("Nx", crt, "", 26, HzUI.Hex("#6E5A8A"), TextAnchor.MiddleCenter, false); _nextStarTxt.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(w - 60, 40), new Vector2(0, -386));`
`SweetTapperGame.Layout.cs:758 / float BuildStarsTab(float w)`	`var info = HzUI.Label("Info", crt, "You keep: chapters, achievements, boosters and your treat.\nStarting over: treats, bakers and recipes.", 24, HzUI.Hex("#6E5A8A"), TextAnchor.MiddleCenter, false);`
`SweetTapperGame.Layout.cs:776 / void UpdateStarsTab()`	`if (_reopenBtn) _reopenBtn.GetComponent<Image>().color = gain > 0 ? Lilac : HzUI.Hex("#C9BEDB");`
`SweetTapperGame.Layout.cs:845 / void RefreshHeat(bool force)`	`_heatFill.color = hot ? Fire : HzUI.Hex("#FFB36B");`
`SweetTapperGame.Layout.cs:872 / void RefreshPills(bool force)`	`case 2: img.color = HzUI.Hex("#F2A93C"); _pillTxt[i].text = "FRENZY x" + D.tuning.frenzyMult.ToString("0", Inv) + " " + sec.ToString(Inv) + "s"; break;`
`SweetTapperGame.Popups.cs:37 / HzPopup StPopup(string title, string message, Color accent, Sprite art, string subtitle, params (string label, Color color, Action action)[] buttons)`	`var card = HzUI.Panel("Card", dim.transform, HzUI.Hex("#FFF8EC"), 56);`
`SweetTapperGame.Popups.cs:40 / HzPopup StPopup(string title, string message, Color accent, Sprite art, string subtitle, params (string label, Color color, Action action)[] buttons)`	`var inner = HzUI.Panel("Inner", card.transform, HzUI.Hex("#FDEBD6"), 44); inner.raycastTarget = false;`
`SweetTapperGame.Popups.cs:174 / void ShowStory(StChapter c)`	`ShowTip(c.story + "\n<color=#E2557A>" + E.GoalText(c) + "</color>");`

`SweetTapperGame.Popups.cs:269 / void DoReopen()`	`Say("Grand Reopening! +" + gain.ToString("#,0", Inv) + " Bakery Stars", HzUI.Hex("#E6D7FF"), 2.4f, 0.75f);`
`SweetTapperGame.Popups.cs:287 / void ShowSettingsPopup()`	`("Wipe Save", HzUI.Hex("#C0584A"), ConfirmWipe),`
`SweetTapperGame.Popups.cs:323 / void ConfirmWipe()`	`StPopup("Wipe save?", "This deletes all progress,\nstars and achievements.", HzUI.Hex("#C0584A"), null, null,`
`SweetTapperGame.Popups.cs:324 / void ConfirmWipe()`	`("Wipe everything", HzUI.Hex("#C0584A"), () => { DebugCommand("reset"); Say("Fresh start!", null, 1.5f); }),`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/SweetTapperGame.Popups.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact helper names, unlocks, charges and effects: Resources/sweettapper/Levels/levels.json / boosters. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`StFx.cs:46 / void Init(RectTransform root)`	`var t = HzUI.Label("n", root, "", 60, Color.white); t.horizontalOverflow = HorizontalWrapMode.Overflow;`
`SweetTapperGame.cs:69 / void RebalancedPopup() =>`	`StPopup("Faster baking!", "Chapters are shorter now and new\nbakers, recipes and helpers unlock\nmuch more often. Your bakery,\nstars and helpers are all kept.", Strawberry, StArt.Get("mascot"), "Bun has news!", ("Let's bake!", Good, null));`
`SweetTapperGame.cs:288 / string BuyModeLabel() => _buyMode == 1 ? "x1" : _buyMode == 10 ? "x10" : "MAX";`	`string BuyModeLabel() => _buyMode == 1 ? "x1" : _buyMode == 10 ? "x10" : "MAX";`
`SweetTapperGame.cs:493 / int Arg(int i, int d) => p.Length > i && int.TryParse(p[i], NumberStyles.Integer, Inv, out var v) ? v : d;`	`$"charges={string.Join(",", Array.ConvertAll(S.charges, x => x.ToString(Inv)))} goldens={S.goldens} tab={_tab} buy={BuyModeLabel()} hint={_hintKey} guide={_guide} " +`
`SweetTapperGame.cs:586 / string PopupTitle()`	`return t ? t.GetComponent<Text>().text.Replace(' ', '_') : "popup";`
`SweetTapperGame.Layout.cs:172 / void BuildCounter(float cx, float cy, float w, float fontPx)`	`_treatsTxt = HzUI.Label("Treats", grp, "0", (int)fontPx, Color.white);`

SweetTapperGame.Layout.cs:175 / void BuildCounter(float cx, float cy, float w, float fontPx)	_cpsTxt = HzUI.Label("Cps", grp, "", (int)(fontPx * 0.4f), HzUI.Hex("#FFE9C9"), TextAnchor.MiddleCenter, false);
SweetTapperGame.Layout.cs:196 / void BuildGoalCard(float cx, float cy, float w, float h)	_goalChapterTxt = HzUI.Label("Ch", card.transform, "", (int)(h * 0.2f), Caramel, TextAnchor.MiddleLeft);
SweetTapperGame.Layout.cs:198 / void BuildGoalCard(float cx, float cy, float w, float h)	_goalTxt = HzUI.Label("Goal", card.transform, "", (int)(h * 0.235f), Ink, TextAnchor.MiddleLeft);
SweetTapperGame.Layout.cs:209 / void BuildGoalCard(float cx, float cy, float w, float h)	_goalNumTxt = HzUI.Label("Num", card.transform, "", (int)(h * 0.18f), CocoaSoft, TextAnchor.MiddleRight);
SweetTapperGame.Layout.cs:221 / void BuildPills()	var t = HzUI.Label("T", p.transform, "", 28, Color.white); t.rectTransform.Stretch(4);
SweetTapperGame.Layout.cs:268 / void BuildHeat(float cx, float cy, float w)	_heatLbl = HzUI.Label("L", bar.transform, "OVEN HEAT", 26, new Color(1, 1, 1, 0.9f)); _heatLbl.rectTransform.Stretch();
SweetTapperGame.Layout.cs:289 / void BuildBoosters(float cx, float cy, float w, bool horizontal)	var btn = HzUI.Button("Booster" + b, _layer, null, Color.white, Color.white, () => OnBooster(bb), 40, (int)(size * 0.32f));
SweetTapperGame.Layout.cs:313 / void BuildBoosters(float cx, float cy, float w, bool horizontal)	v.BadgeText = HzUI.Label("N", bdg.transform, "", (int)(size * 0.25f), Color.white); v.BadgeText.rectTransform.Stretch();
SweetTapperGame.Layout.cs:315 / void BuildBoosters(float cx, float cy, float w, bool horizontal)	v.Caption = HzUI.Label("Cap", v.Root, "", 28, Color.white);
SweetTapperGame.Layout.cs:333 / void RefreshBoosters()	v.BadgeText.text = _guide == b ? "!" : c > 0 ? c.ToString(Inv) : "+";
SweetTapperGame.Layout.cs:352 / void UpdateBoosterTimer(int b)	v.Caption.text = act ? sec.ToString(Inv) + "s" : E.BoosterUnlocked(b) ? bd.name : "Ch. " + bd.unlockChapter.ToString(Inv);
SweetTapperGame.Layout.cs:370 / void BuildSheet(float x, float y, float w, float h, float pad, bool portrait)	var tb = HzUI.Button("Tab" + t, sheet.transform, TabNames[t], HzUI.Hex(TabColors[t]), Color.white, () => OnTabClick(tt), portrait ? 38 : 36, 30);
SweetTapperGame.Layout.cs:380 / void BuildSheet(float x, float y, float w, float h, float pad, bool portrait)	var bm = HzUI.Button("BuyMode", sheet.transform, "", Cocoa, Color.white, CycleBuyMode, 40, 30);

SweetTapperGame.Layout.cs:384 / void BuildSheet(float x, float y, float w, float h, float pad, bool portrait)	var bml = HzUI.Label("Buy", _buyModeBtn, "BUY", 22, HzUI.Hex("#F7D9B9")); bml.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(buyW, 30), new Vector2(0, -16));
SweetTapperGame.Layout.cs:385 / void BuildSheet(float x, float y, float w, float h, float pad, bool portrait)	_buyModeTxt = HzUI.Label("Mode", _buyModeBtn, BuyModeLabel(), 40, Color.white); _buyModeTxt.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(buyW, 50), new Vector2(0, 26));
SweetTapperGame.Layout.cs:542 / float InfoRow(float y, float w, string text)	var t = HzUI.Label("Info", _content, text, 32, CocoaSoft, TextAnchor.MiddleCenter, false);
SweetTapperGame.Layout.cs:560 / float InfoRow(float y, float w, string text)	string artName = tab == 0 ? D.producers[idx].art : (D.upgrades[idx].kind == "prod_mult" ? D.producers[Mathf.Clamp(D.upgrades[idx].target, 0, D.producers.Length - 1)].art : D.upgrades[idx].art);
SweetTapperGame.Layout.cs:574 / float InfoRow(float y, float w, string text)	r.ownedTxt = HzUI.Label("N", ob.transform, "0", 28, Color.white); r.ownedTxt.rectTransform.Stretch();
SweetTapperGame.Layout.cs:578 / float InfoRow(float y, float w, string text)	r.name = HzUI.Label("Name", r.rt, "", (int)(h * 0.27f), Ink, TextAnchor.MiddleLeft);
SweetTapperGame.Layout.cs:581 / float InfoRow(float y, float w, string text)	r.sub = HzUI.Label("Sub", r.rt, "", (int)(h * 0.19f), CocoaSoft, TextAnchor.UpperLeft, false);
SweetTapperGame.Layout.cs:586 / float InfoRow(float y, float w, string text)	r.btn = HzUI.Button("Buy", r.rt, null, Good, Color.white, () => { if (tt == 0) BuyProducer(ii); else BuyUpgrade(ii); }, 40, 28);
SweetTapperGame.Layout.cs:591 / float InfoRow(float y, float w, string text)	r.cost = HzUI.Label("Cost", brt, "", 40, Color.white); r.cost.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(btnW - 70, 56), new Vector2(26, -8));
SweetTapperGame.Layout.cs:594 / float InfoRow(float y, float w, string text)	r.count = HzUI.Label("Cnt", brt, "", 24, new Color(1, 1, 1, 0.9f)); r.count.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(btnW, 30), new Vector2(0, -16));
SweetTapperGame.Layout.cs:616 / void UpdateProducerRow(Row r)	if (!un) { r.name.text = "???"; r.sub.text = "Unlocks in chapter " + p.unlockChapter.ToString(Inv); }

SweetTapperGame.Layout.cs:625 / void UpdateProducerRow(Row r)	r.sub.text = "+" + StNum.Fmt(E.OutputEach(i), true) + "/s each" + (r.owned > 0 ? " - " + StNum.Fmt(E.ProducerCps(i), true) + "/s" : "\n" + p.desc);
SweetTapperGame.Layout.cs:627 / void UpdateProducerRow(Row r)	r.count.text = n > 1 ? "BUY x" + n.ToString(Inv) : "BUY";
SweetTapperGame.Layout.cs:639 / void UpdateUpgradeRow(Row r)	r.sub.text = StEcon.EffectText(D, x) + (avail ? "" : "\nNeeds: " + E.ReqText(u));
SweetTapperGame.Layout.cs:641 / void UpdateUpgradeRow(Row r)	r.count.text = avail ? "LEARN" : "LOCKED";
SweetTapperGame.Layout.cs:659 / float BuildGoalsTab(float w)	var t1 = HzUI.Label("Ch", crt, "CHAPTER " + S.chapter.ToString(Inv) + " - " + c.name, 38, Berry, TextAnchor.MiddleLeft);
SweetTapperGame.Layout.cs:661 / float BuildGoalsTab(float w)	var t2 = HzUI.Label("Story", crt, c.story, 30, Ink, TextAnchor.UpperLeft, false);
SweetTapperGame.Layout.cs:663 / float BuildGoalsTab(float w)	var t3 = HzUI.Label("Goal", crt, E.GoalText(c), 36, Cocoa, TextAnchor.MiddleLeft);
SweetTapperGame.Layout.cs:671 / float BuildGoalsTab(float w)	_gNum = HzUI.Label("Num", crt, "", 30, CocoaSoft, TextAnchor.MiddleRight); _gNum.rectTransform.Place(new Vector2(1, 1), new Vector2(240, 40), new Vector2(-24, -218), new Vector2(1, 1));
SweetTapperGame.Layout.cs:674 / float BuildGoalsTab(float w)	var t4 = HzUI.Label("Reward", crt, reward, 26, CocoaSoft, TextAnchor.MiddleLeft, false); t4.fontStyle = FontStyle.Bold;
SweetTapperGame.Layout.cs:691 / float BuildGoalsTab(float w)	var l = HzUI.Label("N", rt, label, 32, done ? CocoaSoft : Ink, TextAnchor.MiddleLeft); l.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(w * 0.5f, 60), new Vector2(90, 0), new Vector2(0, 0.5f));
SweetTapperGame.Layout.cs:694 / float BuildGoalsTab(float w)	var g = HzUI.Label("G", rt, done ? "Done!" : E.GoalText(ch), 26, CocoaSoft, TextAnchor.MiddleRight, false); g.fontStyle = FontStyle.Bold;
SweetTapperGame.Layout.cs:711 / float BuildGoalsTab(float w)	var n = HzUI.Label("N", rt, a.name, 28, got ? Ink : CocoaSoft, TextAnchor.MiddleLeft); n.rectTransform.Place(new Vector2(0, 1), new Vector2(cw - 90, 42), new Vector2(78, -8), new Vector2(0, 1));

SweetTapperGame.Layout.cs:713 / float BuildGoalsTab(float w)	var dsc = HzUI.Label("D", rt, a.desc, 22, CocoaSoft, TextAnchor.MiddleLeft, false); dsc.rectTransform.Place(new Vector2(0, 0), new Vector2(cw - 90, 40), new Vector2(78, 8), new Vector2(0, 0));
SweetTapperGame.Layout.cs:722 / float Header(float y, float w, string text)	var t = HzUI.Label("H", _content, text, 34, Cocoa, TextAnchor.MiddleLeft);
SweetTapperGame.Layout.cs:736 / void UpdateGoalsTab()	if (key != _gKey) { _gKey = key; _gNum.text = StNum.Fmt(System.Math.Min(prog, c.value)) + " / " + StNum.Fmt(c.value); }
SweetTapperGame.Layout.cs:750 / float BuildStarsTab(float w)	_starsTxt = HzUI.Label("N", crt, "", 52, Plum); _starsTxt.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(w - 60, 64), new Vector2(0, -222));
SweetTapperGame.Layout.cs:751 / float BuildStarsTab(float w)	_starsSubTxt = HzUI.Label("S", crt, "", 30, HzUI.Hex("#6E5A8A"), TextAnchor.MiddleCenter, false); _starsSubTxt.fontStyle = FontStyle.Bold;
SweetTapperGame.Layout.cs:753 / float BuildStarsTab(float w)	_reopenGainTxt = HzUI.Label("G", crt, "", 38, Berry); _reopenGainTxt.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(w - 60, 52), new Vector2(0, -338));
SweetTapperGame.Layout.cs:754 / float BuildStarsTab(float w)	_nextStarTxt = HzUI.Label("Nx", crt, "", 26, HzUI.Hex("#6E5A8A"), TextAnchor.MiddleCenter, false); _nextStarTxt.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(w - 60, 40), new Vector2(0, -386));
SweetTapperGame.Layout.cs:755 / float BuildStarsTab(float w)	var b = HzUI.Button("Reopen", crt, "Grand Reopening", Lilac, Color.white, ShowReopenPopup, 44, 36);
SweetTapperGame.Layout.cs:758 / float BuildStarsTab(float w)	var info = HzUI.Label("Info", crt, "You keep: chapters, achievements, boosters and your treat.\nStarting over: treats, bakers and recipes.", 24, HzUI.Hex("#6E5A8A"), TextAnchor.MiddleCenter, false);
SweetTapperGame.Layout.cs:772 / void UpdateStarsTab()	_starsTxt.text = "Bakery Stars: " + S.stars.ToString("#,0", Inv);

SweetTapperGame.Layout.cs:773 / void UpdateStarsTab()	_starsSubTxt.text = "Each star: +" + Mathf.RoundToInt((float)(D.tuning.starBonus * 100)) + "% taps and production (now +" + StNum.Fmt(S.stars * D.tuning.starBonus * 100) + "%)";
SweetTapperGame.Layout.cs:774 / void UpdateStarsTab()	_reopenGainTxt.text = gain > 0 ? "Reopen now for +" + gain.ToString("#,0", Inv) + " stars" : "Bake more to earn stars";
SweetTapperGame.Layout.cs:775 / void UpdateStarsTab()	_nextStarTxt.text = "Next star at " + StNum.Fmt(E.NextStarAt) + " treats baked in total (now " + StNum.Fmt(S.allBaked) + ")";
SweetTapperGame.Layout.cs:810 / void RefreshCounter(bool force)	if (force || k2 != _cpsKey) { _cpsKey = k2; _cpsTxt.text = cps > 0 ? StNum.Fmt(cps, true) + " treats per second" : "tap the treat to bake!"; }
SweetTapperGame.Layout.cs:820 / void RefreshGoalCard(bool force)	_goalChapterTxt.text = "CHAPTER " + S.chapter.ToString(Inv) + " - " + c.name.ToUpperInvariant();
SweetTapperGame.Layout.cs:828 / void RefreshGoalCard(bool force)	if (key != _goalKey) { _goalKey = key; _goalNumTxt.text = StNum.Fmt(prog) + " / " + StNum.Fmt(c.value); }
SweetTapperGame.Layout.cs:846 / void RefreshHeat(bool force)	_heatLbl.text = hot ? "HOT STREAK x" + D.tuning.streakMult.ToString("0", Inv) : "OVEN HEAT";
SweetTapperGame.Layout.cs:870 / void RefreshPills(bool force)	case 0: img.color = Fire; _pillTxt[i].text = "HOT STREAK " + sec.ToString(Inv) + "s"; break;
SweetTapperGame.Layout.cs:871 / void RefreshPills(bool force)	case 1: img.color = HzUI.Hex(D.boosters[BRush].color); _pillTxt[i].text = "SUGAR RUSH " + sec.ToString(Inv) + "s"; break;
SweetTapperGame.Layout.cs:872 / void RefreshPills(bool force)	case 2: img.color = HzUI.Hex("#F2A93C"); _pillTxt[i].text = "FRENZY x" + D.tuning.frenzyMult.ToString("0", Inv) + " " + sec.ToString(Inv) + "s"; break;
SweetTapperGame.Layout.cs:873 / void RefreshPills(bool force)	case 3: img.color = HzUI.Hex(D.boosters[BBot].color); _pillTxt[i].text = "WHISK BOT " + sec.ToString(Inv) + "s"; break;

SweetTapperGame.Popups.cs:64 / HzPopup StPopup(string title, string message, Color accent, Sprite art, string subtitle, params (string label, Color color, Action action)[] buttons)	var t = HzUI.Label("Title", ribbon.transform, title, 64, Color.white); t.rectTransform.Stretch(8);
SweetTapperGame.Popups.cs:70 / HzPopup StPopup(string title, string message, Color accent, Sprite art, string subtitle, params (string label, Color color, Action action)[] buttons)	var st = HzUI.Label("Sub", card.transform, subtitle, 40, Caramel); st.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(cw - 80, 50), new Vector2(0, y - 25));
SweetTapperGame.Popups.cs:75 / HzPopup StPopup(string title, string message, Color accent, Sprite art, string subtitle, params (string label, Color color, Action action)[] buttons)	p.Message = HzUI.Label("Msg", card.transform, message, 40, Ink, TextAnchor.MiddleCenter, false);
SweetTapperGame.Popups.cs:114 / void StartChapter(bool afterComplete)	Enqueue(() => StPopup("Sweet Tapper", "Hi, I'm Bun! Let's turn this little\nkitchen into the sweetest bakery\nin the universe.", Strawberry, StArt.Get("mascot"), "Welcome to the bakery!", ("Let's bake!", Good, null)));
SweetTapperGame.Popups.cs:149 / void OnChapterComplete(StChapter c, int number, double reward, int prevStage, int prevArea)	StPopup("Chapter " + number + " complete!", string.Join("\n", lines), Strawberry, StArt.Get("mascot"), c.name, ("Next chapter", Good, null));
SweetTapperGame.Popups.cs:169 / void EvolvePopup()	StPopup("Your treat evolved!", "Every tap now bakes x" + mult.ToString("0.#", Inv) + " as much!", Caramel, StArt.Get("tap_" + st), D.tuning.stageNames[st], ("Yummy!", Good, null));
SweetTapperGame.Popups.cs:184 / void ShowChapterPopup()	StPopup("Chapter " + S.chapter, Wrap(c.story, 34) + "\n\nGoal: " + E.GoalText(c) + "\nReward: " + reward, Caramel, StArt.Get("mascot"), c.name, ("Let's bake!", Good, null));
SweetTapperGame.Popups.cs:214 / void UnlockPopup(int b)	StPopup("New helper: " + bd.name + "!", bd.desc + "\nYou get " + bd.freeCharges + " free!", HzUI.Hex(bd.color), StArt.Get(bd.art), null,
SweetTapperGame.Popups.cs:224 / void OfferCharge(int b)	StPopup("+1 " + bd.name, "Out of " + bd.name + ".\nWatch a short ad for one more?", HzUI.Hex(bd.color), StArt.Get(bd.art), null,

`SweetTapperGame.Popups.cs:238 / void WelcomeBack(double amount, double away, double counted)`	`StPopup("Welcome back!", "You were away " + StNum.Time(away) + ".\nYour bakers baked\n" + StNum.Fmt(amount) + " treats!" + capped, Caramel, StArt.Get("mascot"), null,`
`SweetTapperGame.Popups.cs:258 / void ShowReopenPopup()`	`StPopup("Grand Reopening?", "Renovate the bakery and earn\n+" + gain.ToString("#,0", Inv) + " Bakery Stars (+" + pct + "% each).\n\nTreats, bakers and recipes start over.\nChapters, achievements, helpers\nand your treat are kept.",`
`SweetTapperGame.Popups.cs:281 / void ShowSettingsPopup()`	`var p = StPopup("Settings", "", Cocoa, null, null,`
`SweetTapperGame.Popups.cs:286 / void ShowSettingsPopup()`	`("How to Play", Strawberry, () => HelpPage(0)),`
`SweetTapperGame.Popups.cs:297 / void ShowSettingsPopup()`	`parts.onClick.AddListener(() => { bool on = HzSave.GetInt("sweettapper_particles", 1) == 0; HzSave.SetInt("sweettapper_particles", on ? 1 : 0); if (_fx) _fx.Enabled = on; parts.SetText(ParticleLabel()); HzAudio.Play(StSfx.Toggle, 0.6f); });`
`SweetTapperGame.Popups.cs:300 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`SweetTapperGame.Popups.cs:301 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`SweetTapperGame.Popups.cs:302 / static string ParticleLabel() => HzSave.GetInt("sweettapper_particles", 1) == 1 ? "Sprinkles: ON" : "Sprinkles: OFF";`	`static string ParticleLabel() => HzSave.GetInt("sweettapper_particles", 1) == 1 ? "Sprinkles: ON" : "Sprinkles: OFF";`
`SweetTapperGame.Popups.cs:317 / void ShowStats()`	`var p = StPopup("Bakery Stats", s, Caramel, null, null, ("Close", Good, null));`
`SweetTapperGame.Popups.cs:323 / void ConfirmWipe()`	`StPopup("Wipe save?", "This deletes all progress,\nstars and achievements.", HzUI.Hex("#C0584A"), null, null,`
`SweetTapperGame.Popups.cs:331 / void ConfirmWipe()`	`"Recipes are one-time upgrades\nthat multiply your baking.\nHelpers below the treat give\nshort, sweet boosts.",`

`SweetTapperGame.Popups.cs:339 / void HelpPage(int i)`	`StPopup($"How to Play {i + 1}/{HelpPages.Length}", HelpPages[i], Strawberry, i == 0 ? StArt.Get("mascot") : null, null,`
`SweetTapperGame.Popups.cs:340 / void HelpPage(int i)`	`(last ? "Got it" : "Next", Good, last ? (Action)null : () => HelpPage(i + 1)));`
`SweetTapperGame.Popups.cs:357 / void ShowHint(string key, bool resume = false)`	`case "hire": SetTab(0, false); text = "Hire a Whisk Mouse: bakers bake\ntreats for you every second!"; target = FirstRowButton(); break;`
`SweetTapperGame.Popups.cs:374 / void MoveHintToRow()`	`if (_tipText) _tipText.text = "Learn a recipe to bake faster!";`
`SweetTapperGame.Popups.cs:375 / void MoveHintToRow()`	`HzTween.Delay(0.05f, () => { if (_hintKey == "upgrades") PointAt(FirstRowButton()); });`
`SweetTapperGame.Popups.cs:404 / void ShowTip(string text)`	`_tipText = HzUI.Label("T", _tip, text, 34, Ink, TextAnchor.MiddleLeft, false);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/sweettapper through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/sweettapper/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/sweettapper/Music/bakery_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.sweettapper`, version, scene orientation and signing. The build command writes `Builds/Android/sweettapper.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/sweettapper/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.